AI PR Review — how the structure works

Layered AI reviewers that **don't overlap**, posting an inline review like Bugbot. **Advisory, not a gate:** humans still own approval and merge.

✓ Advisory — never approves, never merges ✓ ZDR / self-host (Mozilla) ⚑ DRAFT — local, not wired

branch `deep-review-tooling`

1 The lineup — who catches what

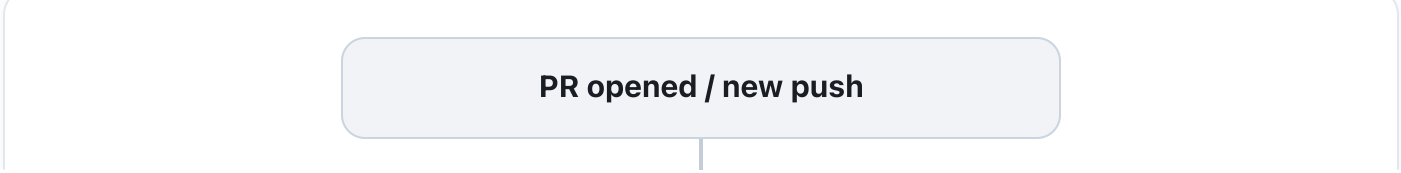
An independent 146-PR test showed AI reviewers **complement rather than duplicate** — findings largely unique to a single reviewer. It's not redundancy — they're different axes. Each layer has its job.

Layer	Tool	Catches	Runs where	Gates?
BUGS	**Cursor Bugbot**	Correctness bugs: races, null, permission, stale refs	Cursor SaaS	no
SAST	**GitHub Advanced Security** (CodeQL + Semgrep)	Vulnerabilities, injection, secrets	GitHub	no*
DESIGN	**thunder-deep-review** *(ours)*	Architecture, house conventions, intent/docs, readability — what bug bots don't see	Our CI (Bedrock/ZDR)	no
OWNER	**Human**	Subjective design calls + the final decision	—	YES

* GHAS can be set to block on *critical* via branch protection — independent of this structure. CodeRabbit is intentionally out of scope for now (easy to add back later).

2 The end-to-end flow

On a PR (open or push), A and B run. B only runs **after** A and reports only what A did **not** catch. At the end, the human sees everything and decides.



RUN IN PARALLEL

Job A — external bot (SaaS)

Cursor Bugbot · GHAS. Run on their own infra, post their **own** comments/checks. We don't orchestrate them.

Job B — thunder-deep-review (our CI)

1. **Debounce 60s** — rapid pushes cancel for free
2. **Wait for A** — poll checks until done (or timeout)
3. **Skip-list** — read A's findings to avoid repeats
4. **Review** the diff — only what A missed
5. **Post an inline review** per line (Bugbot-style)

👤 **Human reviews everything → approves + merges**

Branch protection untouched. No bot approves or merges.

external bot (SaaS) ours (CI / ZDR) human trigger

3 Why B "waits" for A — without `needs:`

The problem

Bugbot **isn't a job in our workflow** — it's an external app. `needs:` only chains internal jobs. We can't declare a dependency on it.

The solution

B **polls the Checks API** until A's checks are `completed` (any conclusion) or the **timeout** hits. If A fails, B still runs. It identifies bots by numeric `app.id` (not name, which drifts).

Orchestration is **deterministic code** (`scripts/review-orchestrator.mjs`), not model tool-calls. The model only emits JSON findings; all GitHub I/O is in the code:

Step	What	Touches creds?
`pre`	poll the bots → build skip-list → compute deep-mode	**no (no AWS)**
`diff`	`git diff base..head` → file (no `Bash` in the model)	**no**
`model`	read diff + skip-list → emit **findings JSON** (read-only)	OIDC→Bedrock

`post`	validate JSON → dedup vs open threads → resolve fixed → post **inline review**	**no**

4 The review — inline, per line (Bugbot-style)

B does **not** post a loose comment. It posts a **PR review with comments anchored to `file:line`** — just like Bugbot. Each finding sits on the exact line, with severity and a fix. `event=COMMENT` (never approve/request-changes → never blocks).

What it posts

- 1 **review** with N **inline comments** (`path` + `line`)
- each: severity (blocking/convention/nit) + fix + cited rule
- anchored to `commit_id` = the reviewed head SHA
- nits grouped/capped so it stays clean

On a new push (convergence)

- re-reviews against the new head
- posts only the **new** findings (dedup vs its own open threads)
- finding **fixed** → **resolves its own thread** (outdated)
- doesn't re-post what's already open

State lives **per thread**, not in one editable post: B manages **its own threads** (resolves when the code is fixed), exactly like Bugbot. It only resolves B's findings; Bugbot handles its own. Dedup is **best-effort** (semantic) — it may occasionally repeat; accepted, not claimed as "zero overlap".

5 Inside thunder-deep-review

A read-only skill that reproduces "the house bar". For high recall it runs as a **fan-out** of specialist reviewers + a merge.

~63%
recall vs an exacting human reviewer (deep mode)

75% / 65%
docs-intent / correctness (where it matters most)

0
references to people (it's the bar, not the person)

7 broad lanes

- `arch` architecture / abstraction / coupling
- `conv` conventions + React (CLAUDE.md)
- `corr` correctness / logic
- `err` error handling + tests

6 micro-specialists (deep mode)

- `µ` dead-code / dead guards
- `µ` async hygiene (.then, fire-and-forget)
- `µ` naming / casing
- `µ` typos / JSDoc

`docs` docs-intent / completeness

`sec` security / privacy / perf

`read` readability / simplification

`μ` simplify / restructure

`μ` module-surface / DRY

+ dispatches the **powersync-sync-reviewer** subagent when the diff touches a synced table (catches the "one table synced, its sibling not" class).

6 Automated vs human

🤖 Automated

- Find bugs, vulns, design/convention/docs issues
- Post them as an inline review
- Re-run + resolve on push
- Non-blocking (advisory)

👤 Human (unchanged)

- Read the findings
- **Approve** the PR
- **Merge**
- Subjective design decisions

Deliberate choice: we dropped auto-merge/auto-approval (no safety net + attack surface). The bot **adds signal**; people decide.

7 Security & privacy

- **Advisory only** — `event=COMMENT`, never approve/request-changes/merge. Branch protection untouched.
- **No forks** — job gated to same-repo PRs; the runner never executes untrusted code. No `pull_request_target`.
- **ZDR** — Claude via Bedrock/Vertex on Mozilla infra (OIDC, no static keys). Code stays in.
- **Read-only** — model gets only `Read/Grep/Glob`, no `Bash`; diff pre-computed (closes prompt-injection).
- **Hardened** — least-privilege, SHA-pinned actions, `timeout-minutes`, ephemeral runner.

Status: draft reviewed by 3 models (GLM-5.2 + MiMo + codex). All local in `drafts/bot-review/` on the `deep-review-tooling` worktree — **nothing wired, nothing committed**. Before going live, 8 items must be resolved

(action input names, SHAs, the bot's `app.id`, OIDC role, model id, runner, diff fetch, CODEOWNERS) — listed in `README.md`.